COMP3334 - Network Security Course Project Report

Obsidian: End-to-End Encrypted Instant Messaging System

Team ID: [Team28]
Submission Date: April 2026

Team Members:
FENG Luquan: 23100577D
YANG Shu: 23098979D
LIU Chen: 22100974D
DU Haoqian: 23098841D
Du Haoyang: 23100929D

Table of Contents

1. Abstract
2. Introduction
3. Threat Model and Assumptions
4. System Architecture
5. Design Specification
6. Security Analysis
7. Testing and Evaluation
8. Conclusion

---

1. Abstract

This project implements an end-to-end encrypted instant messaging system named **Obsidian**, designed to provide a complete secure communication prototype. The system utilizes modern cryptographic techniques to ensure message content remains confidential to the server during transmission and storage.

Core Technical Features:

- **End-to-End Encryption**: X25519 elliptic curve key exchange + HKDF-SHA256 key derivation + AES-GCM authenticated encryption
- **Dual Authentication**: Password authentication + TOTP two-factor authentication

- **Secure Transmission**: HTTPS/WSS TLS encrypted transmission
- **Trust Management**: TOFU (Trust On First Use) trust model
- **Attack Protection**: Replay attack prevention, man-in-the-middle attack protection

Key Achievements:

- Complete client-server architecture prototype
- Support for 1-on-1 real-time encrypted communication
- Comprehensive user management and social relationship system
- Detailed security testing and validation

---

2. Introduction

2.1 Project Background

Traditional instant messaging systems often have security vulnerabilities:

- **Server Access to Plaintext**: Most commercial IM service providers can access user message content
- **Privacy Leakage Risks**: User data may be obtained or misused by third-party organizations
- **Security Threats**: Face network eavesdropping, man-in-the-middle attacks, replay attacks, etc.

End-to-end encryption technology is the key solution to these problems, ensuring that only communicating parties can access message content, even service providers cannot decrypt.

2.2 Project Objectives

The goal of this project is to implement an academic-grade end-to-end encrypted instant messaging system, specifically including:

1. **Technical Validation**: Verify the application of modern cryptographic protocols in real systems
2. **Security Assurance**: Implement the security goal that servers cannot obtain plaintext
3. **Complete Prototype**: Provide a fully functional instant messaging prototype

2.3 Existing Solutions Analysis

Solution	Advantages	Disadvantages
Signal Protocol	Forward secrecy, multi-device support	Complex implementation
WhatsApp E2EE	Large user base, good usability	Closed source, metadata leakage

Solution	Advantages	Disadvantages
Telegram Secret Chat	End-to-end encryption	Limited to 1-on-1, no groups
This Project	**Academic transparency, complete implementation**	**Single device, static keys**

---

3. Threat Model and Assumptions

3.1 Threat Model

```
Attacker Capability Analysis:
├── Network Attacker
│   ├── Eavesdrop network traffic [Protected] (by TLS+E2EE)
│   ├── Modify network data [Protected] (by digital signatures)
│   └── Replay attacks [Protected] (by replay tokens)
├── Server Attacker
│   ├── Obtain server data [Partial] (only ciphertext)
│   ├── Modify server logic [Assumption] (assume server code is trustworthy)
│   └── Inject malicious code [Assumption] (assume server environment is
secure)
└── Client Attacker
    ├── Local data access [Assumption] (assume client environment is secure)
    ├── Malware infection [Out of scope] (beyond project scope)
    └── Physical device acquisition [Assumption] (assume device is secure)
```

3.2 Security Assumptions

Trusted Assumptions:

- Server code is not tampered with and behaves correctly
- Client runtime environment is secure, no malware
- Cryptographic library implementation is correct
- Operating system random number generator quality is reliable
- Users can correctly verify peer fingerprints

Untrusted Assumptions:

- Network transmission may be eavesdropped or tampered with
- Server may be physically accessed by attackers
- User devices may be lost or stolen

3.3 Protection Objectives

Security Goal	Specific Implementation	Protection Level
Confidentiality	End-to-end encryption of message content	Full protection
Integrity	AES-GCM authenticated encryption	Full protection
Authentication	Public key fingerprint verification	Full protection
Replay Resistance	Replay token mechanism	Full protection

---

4. System Architecture

4.1 Overall Architecture Diagram

```
┌──────────────────┐    HTTPS/WSS    ┌──────────────────┐
│   Client A       │ ←──────────────→ │   Server         │
│                  │                  │                  │
│ ┌──────────────┐ │                  │ ┌──────────────┐ │
│ │ CLI Interface│ │                  │ │ FastAPI      │ │
│ └──────────────┘ │                  │ └──────────────┘ │
│ ┌──────────────┐ │                  │ ┌──────────────┐ │
│ │ E2EE Encryption│ │                  │ │ SQLite DB    │ │
│ └──────────────┘ │                  │ └──────────────┘ │
│ ┌──────────────┐ │                  │ ┌──────────────┐ │
│ │ Local State  │ │                  │ │ WebSocket    │ │
│ └──────────────┘ │                  │ └──────────────┘ │
└──────────────────┘                  └──────────────────┘
         │                                    │
         │                                    │
         └────────────────────────────────────┘
                    Encrypted Message Flow
```

4.2 Data Flow Design

4.2.1 Message Sending Flow

```
1. User inputs plaintext message
   ↓
2. Client obtains recipient's public key (GET /identity-key/{username})
   ↓
3. Local X25519 key exchange
   ↓
4. HKDF-SHA256 key derivation
   ↓
5. AES-GCM encrypt message + AAD binding
   ↓
```

```
6. Build JSON ciphertext envelope
   ↓
7. HTTPS send to server (POST /messages/send)
   ↓
8. Server stores ciphertext to SQLite
   ↓
9. WebSocket push to recipient
```

4.2.2 Message Receiving Flow

```
1. Receive WebSocket push event
   ↓
2. Obtain sender's public key (local cache or server query)
   ↓
3. Local X25519 key exchange
   ↓
4. HKDF-SHA256 key derivation
   ↓
5. AES-GCM decrypt message + AAD verification
   ↓
6. Display plaintext message to user
   ↓
7. Send read receipt (POST /messages/ack)
```

4.3 Technology Stack

Component	Technology Choice	Version	Purpose
Backend Framework	FastAPI	≥0.115.0	REST API and WebSocket
Database	SQLite	3.x	User data and message storage
Crypto Library	cryptography	≥42.0.0	Cryptographic primitives implementation
HTTP Client	httpx	≥0.27.0	Asynchronous HTTP requests
WebSocket	websockets	≥12.0	Real-time communication
Password Hashing	argon2-cffi	≥23.1.0	Secure password storage
Data Validation	pydantic	≥2.0.0	API data validation

---

5. Design Specification

5.1 Protocol Design

5.1.1 Session Establishment Protocol

Phase 1: User Registration

```
POST /register
Content-Type: application/json

{
  "username": "alice",
  "password": "StrongPass123"
}

Response:
{
  "user_id": 1,
  "username": "alice",
  "otp_secret": "JBSWY3DPEHPK3PXP...",
  "otp_uri": "otpauth://totp/COMP3334-IM:alice?secret=...",
  "message": "registration successful"
}
```

Phase 2: Dual Authentication Login

```
# Step 1: Password authentication
POST /login/password
{
  "username": "alice",
  "password": "StrongPass123"
}

Response:
{
  "challenge_token": "eyJ...",
  "expires_at": "2026-04-01T15:17:00Z"
}

# Step 2: OTP verification
POST /login/otp
{
  "challenge_token": "eyJ...",
  "otp_code": "123456"
}

Response:
{
  "access_token": "eyJ...",
```

```
  "token_type": "bearer",
  "expires_at": "2026-04-02T03:17:00Z"
}
```

Phase 3: Key Management

```
POST /identity-key
Authorization: Bearer <access_token>

{
  "device_id": "cli-device-1",
  "public_key": "base64 encoded X25519 public key"
}

Response:
{
  "ok": true,
  "device_id": "cli-device-1",
  "fingerprint": "0123abcd89ef5678..."
}
```

Phase 4: Trust Establishment

```
GET /identity-key/bob
Authorization: Bearer <access_token>

Response:
{
  "username": "bob",
  "device_id": "cli-device-1",
  "public_key": "base64 encoded public key",
  "fingerprint": "89ef56780123abcd...",
  "created_at": "2026-04-01T03:00:00Z"
}
```

5.1.2 Message Format Design

Encrypted Envelope Format:

```
{
  "v": 2,
  "alg": "x25519-hkdf-sha256-aesgcm",
  "ciphertext": "base64 encoded AES-GCM ciphertext",
  "nonce": "base64 encoded 12-byte random nonce",
  "salt": "base64 encoded 16-byte HKDF salt",
  "replay_token": "32-digit hexadecimal string",
  "sender_device_id": "cli-device-1",
```

```
  "ttl_seconds": 3600
}
```

Additional Authenticated Data (AAD):

```
{
  "from_username": "alice",
  "to_username": "bob",
  "message_type": "e2ee_text",
  "sender_device_id": "cli-device-1",
  "replay_token": "abc123def456...",
  "ttl_seconds": 3600
}
```

5.1.3 Message Transmission Protocol

Send Message:

```
POST /messages/send
Authorization: Bearer <access_token>

{
  "to_username": "bob",
  "content": "JSON format encrypted envelope",
  "message_type": "e2ee_text"
}

Response:
{
  "ok": true,
  "message": "submitted",
  "data": {
    "message_id": 7,
    "conversation_id": 1,
    "from_username": "alice",
    "to_username": "bob",
    "content": "local plaintext display",
    "message_type": "e2ee_text",
    "status": "sent",
    "created_at": "2026-04-01T03:30:00Z"
  }
}
```

5.2 Cryptographic Primitive Selection

5.2.1 Key Exchange: X25519

Selection Rationale:

- **Security**: Provides 128-bit security level, resistant to quantum computing attacks
- **Efficiency**: Fast computation speed, suitable for mobile devices and embedded systems
- **Standardization**: RFC 7748 standard, widely supported and validated
- **Implementation Quality**: `cryptography` library provides secure and reliable implementation

Technical Parameters:

```
# Key generation
private_key = X25519PrivateKey.generate()
public_key = private_key.public_key()

# Key exchange
shared_secret = private_key.exchange(peer_public_key)
# Output: 32-byte shared secret
```

5.2.2 Key Derivation: HKDF-SHA256

Selection Rationale:

- **Security**: HMAC based on SHA-256, strong collision resistance
- **Flexibility**: Can output arbitrary length key material
- **Standardization**: RFC 5869 standard, cryptographic community consensus
- **Applicability**: Suitable for deriving multiple subkeys from shared secret

Technical Implementation:

```
hkdf = HKDF(
    algorithm=hashes.SHA256(),
    length=32,                    # AES-256 key length
    salt=salt,                    # 16-byte random salt
    info=b'comp3334-im-e2ee-v1'   # Application-specific info
)
content_key = hkdf.derive(shared_secret)
```

5.2.3 Symmetric Encryption: AES-GCM

Selection Rationale:

- **Authenticated Encryption**: Provides both confidentiality and integrity protection
- **Performance Advantage**: Modern CPUs provide hardware acceleration support
- **Standardization**: NIST SP 800-38D standard
- **Security**: Resistant to chosen ciphertext attacks and other advanced attacks

Technical Implementation:

```
aesgcm = AESGCM(content_key)
ciphertext = aesgcm.encrypt(
    nonce=nonce,                    # 12-byte random nonce
    data=plaintext.encode('utf-8'), # Plaintext data
    associated_data=aad             # Additional authenticated data
)
```

5.2.4 Cryptographic Library Version and Dependencies

Core Dependencies:

```
cryptography>=42.0.0
├── Supports X25519 elliptic curve key exchange
├── Supports HKDF-SHA256 key derivation
├── Supports AES-GCM authenticated encryption
├── Provides secure random number generation
└── Complies with FIPS 140-2 standard

argon2-cffi>=23.1.0
├── Memory-hard password hashing function
├── Resistant to GPU/ASIC brute force attacks
└── OWASP password hashing recommended algorithm
```

---

6. Security Analysis

6.1 Server Inability to Access Plaintext Analysis

6.1.1 Encryption Location and Flow

```
Client Encryption Flow:
Plaintext Message
    ↓ [Local Encryption]
X25519 Key Exchange → HKDF Key Derivation → AES-GCM Encryption
    ↓
JSON Ciphertext Envelope
    ↓ [HTTPS Transmission]
Server (Only stores ciphertext)
```

Key Security Guarantees:

1. **Keys Never Leave Client**: Private keys only stored in `client/client_state.json`
2. **Server Cannot Decrypt**: Lacks private keys for X25519 key exchange

3. **Transport Layer Protection**: TLS prevents network eavesdropping and man-in-the-middle attacks

6.1.2 Server Storage Content Analysis

Database Message Table Structure:

```sql
CREATE TABLE messages (
    message_id INTEGER PRIMARY KEY,
    conversation_id INTEGER,
    from_username TEXT,
    to_username TEXT,
    content TEXT,              -- JSON ciphertext envelope
    message_type TEXT,         -- 'e2ee_text'
    status TEXT,
    is_offline_queued BOOLEAN,
    is_read BOOLEAN,
    created_at TEXT,
    delivered_at TEXT,
    read_at TEXT
);
```

Server Perspective Data:

```json
{
  "from_username": "alice",           // Plaintext metadata
  "to_username": "bob",               // Plaintext metadata
  "content": "eyJ2IjoywiJhbGci...=",  // Ciphertext envelope (cannot
decrypt)
  "message_type": "e2ee_text",        // Plaintext type identifier
  "created_at": "2026-04-01T03:30:00Z", // Plaintext timestamp
  "status": "delivered",              // Plaintext status information
  "is_read": true                     // Plaintext read status
}
```

6.1.3 Decryption Capability Verification

Server Decryption Attempt (Fails):

```python
# Server-side pseudocode - cannot succeed
def server_decrypt_attempt():
    encrypted_content = get_message_from_db()
    envelope = json.loads(encrypted_content)

    # Lacks private key, cannot perform key exchange
    try:
        shared_secret = ???  # Server doesn't have Alice's private key
```

```
        content_key = hkdf_derive(shared_secret, envelope['salt'])
        plaintext = aesgcm_decrypt(envelope, content_key)
    except Exception as e:
        return "Decryption failed: missing necessary private key material"
```

6.2 Metadata Leakage Analysis

6.2.1 Types of Leaked Metadata

User Relationship Graph:

```
Server-Known Social Relationships:
├── Contact Relationships
│   ├── Alice ↔ Bob (Contacts)
│   ├── Alice ↔ Carol (Contacts)
│   └── Bob ↔ Carol (Non-contacts)
├── Friend Request History
│   ├── Alice → Bob (Accepted)
│   ├── Carol → Alice (Rejected)
│   └── Bob → Alice (Pending)
└── Block Relationships
    ├── Alice blocked Dave
    └── Bob blocked Eve
```

Communication Pattern Analysis:

```
Inferable Behavior Patterns:
├── Time Patterns
│   ├── Alice active 9-18 daily
│   ├── Bob mainly online evenings
│   └── Weekend message frequency increases 50%
├── Relationship Strength
│   ├── Alice-Bob: Daily 10+ messages
│   ├── Alice-Carol: Weekly 2-3 messages
│   └── New contacts: Initial high message frequency
└── Network Topology
    ├── Core users: Connections >20
    ├── Edge users: Connections <5
    └── Bridge nodes: Connecting different communities
```

Technical Metadata:

```
System-Known Information:
├── Account Information
│   ├── Registration time: 2026-03-15
│   ├── Last login: 2026-04-01 15:30
│   └── Device ID: cli-device-1
```

```
├── Message Statistics
│   ├── Sent messages: 1,247
│   ├── Received messages: 1,198
│   ├── Average message length: 45 characters
│   └── Encrypted message size: 280 bytes average
└── Network Information
    ├── IP address: 192.168.1.100
    ├── Geographic location: Hong Kong
    └── Network type: Home broadband
```

6.2.2 Privacy Protection Measures

Implemented Protections:

- **Message Content Encryption**: Core communication content completely confidential
- **Identity Verification**: Public key fingerprint prevents man-in-the-middle attacks
- **Integrity Protection**: AES-GCM prevents message tampering
- **Replay Protection**: Replay token prevents repeated attacks

Unprotected Metadata:

- **Communication Relationships**: Who is communicating with whom
- **Time Information**: Communication timing and frequency
- **Network Topology**: User social network structure
- **Behavior Patterns**: User activity patterns

6.2.3 Privacy Impact Assessment

Low Risk Impact:

- Message content completely confidential
- Cannot perform content censorship
- Protects user speech privacy

Medium Risk Impact:

- Can infer social relationship networks
- Can analyze user behavior patterns
- Possible traffic analysis attacks

High Risk Impact (Not Implemented):

- Lacks metadata obfuscation techniques
- No anonymous communication mechanisms
- No traffic padding protection

6.3 System Limitations

6.3.1 Technical Limitations

Key Management Limitations:

```
Current Limitations:
├── Static Key System
│   ├── No Forward Secrecy
│   ├── No Backward Secrecy
│   ├── No key rotation mechanism
│   └── Key compromise affects all historical messages
├── Single Device Support
│   ├── Only supports cli-device-1
│   ├── No multi-device synchronization
│   ├── No device management mechanism
│   └── Device loss causes key loss
└── Simplified Trust Model
    ├── Only TOFU trust mode
    ├── No manual key verification process
    ├── No trust level management
    └── No key change auditing
```

Storage Security Limitations:

```
Local Storage Issues:
├── Plaintext State Storage
│   ├── client_state.json not encrypted
│   ├── Contains private keys, OTP secrets and other sensitive information
│   ├── No password protection mechanism
│   └── Malware can directly obtain
├── No Secure Deletion
│   ├── Key remnants after account deletion
│   ├── No memory secure cleanup
│   └── No disk secure wiping
└── No Backup Encryption
    ├── State file backup not encrypted
    ├── Cloud sync has leakage risk
    └── No version control security
```

Protocol Function Limitations:

```
Missing Features:
├── Message Features
│   ├── No group message support
│   ├── No file transfer functionality
│   ├── No message recall mechanism
│   └── No message editing functionality
├── User Experience
```

```
│   ├── Complex CLI interface
│   ├── No graphical interface
│   ├── Unfriendly error prompts
│   └── No usage wizard
└── System Management
    ├── No user management interface
    ├── No system monitoring
    ├── No audit logs
    └── No backup and recovery
```

6.3.2 Security Improvement Recommendations

Short-term Improvements:

1. **Local State Encryption**: Use system keychain to encrypt client sensitive data, preventing local file leakage
2. **Enhanced Error Handling**: Provide more friendly security error prompts and user guidance
3. **Memory Security**: Implement memory cleanup and secure release mechanisms for sensitive data

Medium-term Improvements:

1. **Implement Forward Secrecy**: Adopt double ratchet algorithm, generating new temporary keys for each message
2. **Multi-device Support**: Implement pre-key mechanisms and device management, supporting multi-device synchronization
3. **Group Messages**: Extend protocol to support group end-to-end encryption

Long-term Improvements:

1. **Metadata Protection**: Implement onion routing or obfuscation techniques to hide communication relationships and time information
2. **Production Environment Deployment**: Add load balancing, security monitoring, and disaster recovery mechanisms
3. **Performance Optimization**: Optimize cryptographic algorithm implementations, enhance large-scale user support capabilities

---

7. Testing and Evaluation

7.1 Functional Demonstrations

7.1.1 Complete Communication Flow Test

Test Environment:

- Operating System: Windows 11 / Ubuntu 22.04
- Python Version: 3.11+
- Dependency Versions: requirements.txt specified versions

Test Steps:

```
# 1. Start server
$ python -m server.run_tls

# 2. Terminal A: Alice registration and login
$ python client/cli.py https://127.0.0.1:8443
> register alice StrongPass123

> login alice StrongPass123

# 3. Terminal B: Bob registration and login
$ python client/cli.py https://127.0.0.1:8443
> register bob StrongPass123

> login bob StrongPass123
```

```
MINGW64:/c/Users/ymr/Documents/files/polyu/year3sem2/security/Obsidian-A-P...
[push] auth failed: websocket authentication failed (token invalid or expired);
please login again
[push] local session cleared. run: login <username> <password>


im>

im> register alice StrongPass123
{'user_id': 1, 'username': 'alice', 'otp_secret': 'BLWR4Q3PYKZLW4F6KAN4PZXIKZ3HC
45F', 'otp_uri': 'otpauth://totp/COMP3334-IM:alice?secret=BLWR4Q3PYKZLW4F6KAN4PZ
XIKZ3HC45F&issuer=COMP3334-IM', 'message': 'registration successful; save the OT
P secret in your authenticator app'}
Current OTP code: 705362

im> login alice StrongPass123
Using locally stored OTP code for demo: 705362
{'access_token': 'HQsM4NfYFadiyZH5BDQPVLcCIjRBTzQEcf9z-IpnxQg', 'token_type': 'b
earer', 'expires_at': '2026-04-06T03:26:09.037723+00:00'}
E2EE ready on device cli-device-1 (fingerprint 0c80ebb2390f9e5179fd9344ccabd1d43
913c5c8ad9a7f603b40268d79dd66d2)

im>
[push] system: {'message': 'connected as alice'}
```

```
MINGW64:/c/Users/ymr/Documents/files/polyu/year3sem2/security/Obsidian-A-P...
  store-dev-key                # ensures and republishes the real E2EE identity
key for this device
  exit

im> me
error: missing bearer token

im> register bob StrongPass123
{'user_id': 2, 'username': 'bob', 'otp_secret': '3TYYVBRWPWZRQNUV3SBWKCWZJNHYJ47
D', 'otp_uri': 'otpauth://totp/COMP3334-IM:bob?secret=3TYYVBRWPWZRQNUV3SBWKCWZJN
HYJ47D&issuer=COMP3334-IM', 'message': 'registration successful; save the OTP se
cret in your authenticator app'}
Current OTP code: 729078

im> login bob StrongPass123
Using locally stored OTP code for demo: 729078
{'access_token': 'MQjwPYcXG3hDI83Z1NFdBcmiR3yUie8F-14Wa5WtAfU', 'token_type': 'b
earer', 'expires_at': '2026-04-06T03:26:27.692234+00:00'}
E2EE ready on device cli-device-1 (fingerprint 71abc5c5fe6ff1a2760c92c3ab3294a57
f719286c242174685962b00570d3e08)

im>
[push] system: {'message': 'connected as bob'}
```

Alice and Bob register and login

```
# 4. Alice adds friend
> send-request bob

# 5. Bob accepts friend request
> pending
> respond 1 accept

# 6. Alice sends encrypted message
> send bob "Hello Bob! This is a secret message."

# 7. Bob views messages
```

```
> conversations
> open 1
```

Successful sending and receiving:

```
im> send bob hello bob
{'ok': True, 'message': 'submitted', 'data': {'message_id': 2, 'conversation_id'
: 1, 'from_username': 'alice', 'to_username': 'bob', 'content': 'hello bob', 'me
ssage_type': 'e2ee_text', 'ttl_seconds': None, 'expires_at': None, 'status': 'se
nt', 'is_offline_queued': False, 'is_read': False, 'created_at': '2026-04-05T15:
30:40.019969+00:00', 'delivered_at': None, 'read_at': None}}
(E2EE sender fingerprint 0c80ebb2390f9e5179fd9344ccabd1d43913c5c8ad9a7f603b40268
d79dd66d2; trusted peer fingerprint 71abc5c5fe6ff1a2760c92c3ab3294a57f719286c242
174685962b00570d3e08)

im>
[push] your message 2 is delivered
```

```
im>
[push] new message from alice: hello bob
[push] auto-acked message 2 as delivered
```

Two-way message exchange:

```
im>
[push] your message 2 is delivered

[push] new message from bob: "Hi Alice! Got your message!"
[push] auto-acked message 3 as delivered
```

```
im>
[push] new message from alice: hello bob
[push] auto-acked message 2 as delivered
send alice "Hi Alice! Got your message!"
```

Bob offline, Alice sends message to Bob, Bob can see after login:

```
send bob "Bob are you there?"
```

```
im> im> login bob StrongPass123
error: unknown command; type help

im> login bob StrongPass123
Using locally stored OTP code for demo: 585617
{'access_token': 'zopAJssFyaVTHKZLTWvvCzHShkYh_4Jt_illJ-VIdck', 'token_type': 'b
earer', 'expires_at': '2026-04-06T04:09:50.046212+00:00'}
E2EE ready on device cli-device-1 (fingerprint 71abc5c5fe6ff1a2760c92c3ab3294a57
f719286c242174685962b00570d3e08)

im>
[push] new message from alice: "Bob are you there?"
```

Alice blocks Bob, two cannot communicate:

```
im> block bob
{'ok': True, 'message': 'blocked user bob; removed contact relationship'}

im> blocked
{'blocked_users': [{'id': 2, 'username': 'bob', 'created_at': '2026-04-05T16:21:
51.292478+00:00'}]}

im>
```

```
im> send alice "can you see me ?"
error: message blocked by user policy

im>
```

Alice unblocks Bob, then two add friends, after that can send messages to each other:

```
im> Unblock bob
{'ok': True, 'message': 'unblocked user bob'}

im> send-request bob
{'ok': True, 'message': 'friend request sent', 'request_id': 3, 'target_username
': 'bob', 'target_user_id': 2}
(Request #3 sent to user @bob, id=2)

im>
[push] friend request update: {'request_id': 3, 'from_username': 'alice', 'to_us
ername': 'bob', 'status': 'accepted'}

[push] new message from bob: "can you see me now?"
[push] auto-acked message 6 as delivered
```

```
pending 3
{'incoming': [{'request_id': 3, 'status': 'pending', 'created_at': '2026-04-05T1
6:23:17.772502+00:00', 'responded_at': None, 'from_username': 'alice', 'to_usern
ame': 'bob'}], 'outgoing': []}

im> respond 3 accept
{'ok': True, 'message': 'friend request accepted'}

im> send alice "can you see me now?"
{'ok': True, 'message': 'submitted', 'data': {'message_id': 6, 'conversation_id'
: 1, 'from_username': 'bob', 'to_username': 'alice', 'content': '"can you see me
 now?"', 'message_type': 'e2ee_text', 'ttl_seconds': None, 'expires_at': None, '
status': 'sent', 'is_offline_queued': False, 'is_read': False, 'created_at': '20
26-04-05T16:24:04.516019+00:00', 'delivered_at': None, 'read_at': None}}
(E2EE sender fingerprint 71abc5c5fe6ff1a2760c92c3ab3294a57f719286c242174685962b0
0570d3e08; trusted peer fingerprint 0c80ebb2390f9e5179fd9344ccabd1d43913c5c8ad9a
7f603b40268d79dd66d2)

im>
[push] your message 6 is delivered
```

Alice sets 30-second expiration message, Bob can view in history within 30 seconds:

```
send-ttl bob 30 "This message will self-destruct in 30 seconds"
{'ok': True, 'message': 'submitted', 'data': {'message_id': 8, 'conversation_id'
: 1, 'from_username': 'alice', 'to_username': 'bob', 'content': '"This message w
ill self-destruct in 30 seconds"', 'message_type': 'e2ee_text', 'ttl_seconds': 3
0, 'expires_at': '2026-04-05T16:27:48.529713+00:00', 'status': 'sent', 'is_offli
ne_queued': False, 'is_read': False, 'created_at': '2026-04-05T16:27:18.530720+0
0:00', 'delivered_at': None, 'read_at': None}}
(self-destruct after 30 seconds)
(E2EE sender fingerprint 0c80ebb2390f9e5179fd9344ccabd1d43913c5c8ad9a7f603b40268
d79dd66d2; trusted peer fingerprint 71abc5c5fe6ff1a2760c92c3ab3294a57f719286c242
174685962b00570d3e08)

im>
[push] your message 8 is delivered

[expire] self-destruct message 8 expired and will no longer appear in history
```

```
0', 'delivered_at': '2026-04-05T16:24:04.541143+00:00', 'read_at': None}, {'mess
age_id': 8, 'conversation_id': 1, 'from_username': 'alice', 'to_username': 'bob'
, 'content': '"This message will self-destruct in 30 seconds"', 'message_type':
```

After time passes, cannot be seen:

```
5+00:00'}, {'message_id': 6, 'conversation_id': 1, 'from_username': 'bob', 'to
sername': 'alice', 'content': '"can you see me now?"', 'message_type': 'e2ee_t
t', 'ttl_seconds': None, 'expires_at': None, 'status': 'delivered', 'is_offlin
queued': False, 'is_read': False, 'created_at': '2026-04-05T16:24:04.516019+00
0', 'delivered_at': '2026-04-05T16:24:04.541143+00:00', 'read_at': None}], 'ne
_before_id': 1}
```

7.1.2 Encryption Verification Test

Database Verification:

- View server database, verify stored message content is in ciphertext format
- Confirm message type is `e2ee_text`, content field is Base64 encoded encrypted envelope
- Verify ciphertext contains version number, algorithm identifier, ciphertext, nonce, salt and replay_token fields

```
  SELECT id, from_username, to_username, content, message_type FROM messages LIM
             ^--- error here
sqlite> SELECT
   ...>   m.id,
   ...>   u1.username as sender,
   ...>   u2.username as receiver,
   ...>   m.content,
   ...>   m.message_type
   ...> FROM messages m
   ...> JOIN users u1 ON m.sender_id = u1.id
   ...> JOIN users u2 ON m.receiver_id = u2.id
   ...> LIMIT 1;
1|alice|bob|{"alg":"x25519-hkdf-sha256-aesgcm","ciphertext":"Cz61FhtRJ/ceZIHADdJ
L9BLFazQrbw+uYw==","nonce":"3UtgFP2NQymDTKrl","replay_token":"cc81050cdb9c919b70
0a6237883f69c3","salt":"+AyH17+zqH4kxAI9Bcwfog==","sender_device_id":"cli-device
-1","v":2}|e2ee_text
sqlite> |
```

Database cannot see plaintext content

Client Decryption Verification:

- Use client private key to obtain ciphertext from server
- Call decryption function to decrypt
- Verify decrypted plaintext matches original message
- Confirm server cannot directly obtain plaintext content

7.2 Security Test Cases

7.2.1 Security Test Case 1: Replay Attack Protection

Test Objective: Verify replay token mechanism effectively protects against replay attacks

Test Implementation: Based on `ReplayProtectionTests` class in project `tests/test_replay_protection.py`

Test Method:

1. **Environment Setup**: Use `TemporaryDirectory` to create independent test database
2. **User Initialization**: Register Alice and Bob users through `TestClient` and obtain access tokens
3. **E2EE Manager**: Initialize `ClientE2EEManager` for end-to-end encryption operations
4. **Friend Relationship**: Establish contact relationship between Alice and Bob

Key Test Cases:

Test 1: Duplicate Delivery Detection
(`test_duplicate_delivery_on_reconnect_is_detected`)

- Alice sends message to Bob
- Bob receives message through WebSocket for first time
- Simulate receiving same message after reconnection
- Verify system throws `DuplicateDeliveryError` exception

Test 2: Replay Ciphertext Blocking
(`test_replayed_ciphertext_with_new_server_message_id_is_blocked`)

- Alice sends original message
- Attacker inserts same ciphertext into database (simulating replay attack)
- Attempt to decrypt replayed message
- Verify system throws `ReplayAttackError` exception

For detailed test code, please refer to: `tests/test_replay_protection.py` lines 106-185
Test results: All three tests passed

```
$ python -m pytest tests/test_replay_protection.py -v
============================= test session starts ==============================
platform win32 -- Python 3.9.25, pytest-8.4.2, pluggy-1.6.0 -- C:\Users\ymr\anac
onda3\envs\Obsidian\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\ymr\Documents\files\polyu\year3sem2\security\Obsidian-A-Privac
y-Preserving-Instant-Messaging-System-with-Authenticated-Key-Exchange
plugins: anyio-4.12.1
collected 3 items

tests/test_replay_protection.py::ReplayProtectionTests::test_duplicate_delivery_
on_reconnect_is_detected PASSED [ 33%]
tests/test_replay_protection.py::ReplayProtectionTests::test_replayed_ciphertext
_with_new_server_message_id_is_blocked PASSED [ 66%]
tests/test_replay_protection.py::ReplayProtectionTests::test_same_message_can_st
ill_be_opened_from_history PASSED [100%]
```

Test Result Verification:

- `DuplicateDeliveryError` exception correctly thrown
- `ReplayAttackError` exception correctly thrown
- Original message decrypts normally, replayed message is blocked
- System has complete replay attack protection capabilities

Security Significance:

- Prevents attackers from intercepting messages and repeatedly sending them
- Protects communication integrity and freshness
- Prevents message counting and status confusion
- Validates effectiveness of replay token mechanism

7.2.2 Security Test Case 2: TLS Transport Security Test

Test Objective: Verify security of TLS/SSL encrypted transmission

Test Implementation: Based on TLS transport tests in project `tests/test_tls_transport.py`

Test Method:

1. **TLS Server**: Use `_TLSUvicornServer` to start independent TLS test server
2. **Certificate Generation**: Generate development TLS certificates through `ensure_dev_tls_materials()`
3. **Port Management**: Use `_find_free_port()` to dynamically allocate available ports
4. **Client Connection**: Use `httpx` and `WebSocketListener` for secure connection testing

Key Test Components:

TLS Server Management:

- Use uvicorn to configure SSL certificates and keys
- Run TLS server in independent thread
- Support HTTPS and WSS connections

For detailed TLS server implementation, please refer to: `tests/test_tls_transport.py` lines 31-50

Test Flow:

1. **Certificate Preparation**: Generate self-signed CA certificate and server certificate
2. **Server Startup**: Start TLS server on random port
3. **HTTPS Connection**: Client verifies server certificate and establishes HTTPS connection
4. **WSS Connection**: Establish secure WebSocket connection
5. **Data Transmission**: Verify all communications are encrypted through TLS

Security Verification Points:

- **Certificate Verification**: Client correctly verifies server certificate chain
- **Encrypted Transmission**: All HTTP/WebSocket communications encrypted through TLS 1.2+
- **Port Isolation**: Tests use independent ports to avoid conflicts
- **Connection Security**: Prevent man-in-the-middle attacks and eavesdropping

Test Results:

- TLS server successfully starts and listens on specified port
- HTTPS client connection established successfully, certificate verification passed
- WebSocket over TLS (WSS) connection works normally
- All network traffic is encrypted and protected

Both tests passed:

```
$ python -m pytest tests/test_tls_transport.py -v
========================== test session starts ==========================
platform win32 -- Python 3.9.25, pytest-8.4.2, pluggy-1.6.0 -- C:\Users\ymr\anac
onda3\envs\Obsidian\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\ymr\Documents\files\polyu\year3sem2\security\Obsidian-A-Privac
y-Preserving-Instant-Messaging-System-with-Authenticated-Key-Exchange
plugins: anyio-4.12.1
collected 2 items

tests/test_tls_transport.py::TLSTransportTests::test_api_client_rejects_plain_ht
tp_by_default PASSED [ 50%]
tests/test_tls_transport.py::TLSTransportTests::test_https_and_wss_transport_sup
port_full_secure_message_flow PASSED [100%]
```

Security Significance:

- Validates correct implementation of TLS transport layer
- Ensures confidentiality and integrity of client-server communication
- Prevents network-level man-in-the-middle attacks
- Provides secure transmission foundation for end-to-end encryption

---

Appendix A: Project Requirements Checklist (R1-R25)

A.1 User Authentication

(R1) Registration

- Users register with unique username and password
- Passwords hashed using Argon2 with per-user salt
- Rate limiting applied to registration endpoint

(R2) Login with Password + OTP

- Two-factor authentication: password + TOTP
- Session tokens expire after 12 hours
- Tokens bound to authenticated user

(R3) Logout / Session Invalidation

- Users can explicitly logout
- Tokens are removed from server database
- Immediate session invalidation

A.2 Identity & Key Management

(R4) Per-device Identity Keypair

- Each client generates X25519 keypair locally
- Private keys stored only in client state file
- Server stores only public keys for session establishment

(R5) Fingerprint / Verification UI

- SHA-256 hash of public key displayed as fingerprint
- Users can mark contacts as "verified" locally
- Verification status stored in client state

(R6) Key Change Detection

- Client detects when contact's public key changes
- Warning displayed to user about key change
- Policy: Allow with warning, require re-verification

A.3 E2EE 1:1 Messaging

(R7) Secure Session Establishment

- X25519 elliptic curve Diffie-Hellman key exchange
- HKDF-SHA256 for key derivation
- Static keys with per-message salt for security

(R8) Message Encryption and Authentication

- AES-GCM (256-bit) for authenticated encryption
- Metadata bound using Additional Authenticated Data (AAD)
- Includes sender/receiver IDs, message type, replay token

(R9) Replay Protection / De-duplication

- 32-byte replay tokens included in each message
- Client maintains replay cache of seen tokens
- Messages with duplicate tokens are rejected

A.4 Timed Self-Destruct Messages

(R10) TTL / Expiration Policy

- Messages can have configurable TTL (seconds)
- TTL included in authenticated metadata
- Server and client both enforce expiration

(R11) Client Deletion Behavior

- Expired messages removed from UI and local storage
- Automatic cleanup on client startup and message fetch
- Messages with expired TTL not displayed
- Client: `client/cli.py` - message filtering logic
- Display: `client/cli.py` - `open_conversation()` function
- Storage: `client/client_state.json` - periodic cleanup

(R12) Server Storage Behavior (Best-effort)

Implementation Summary:

- Server deletes expired ciphertext from database
- Best-effort cleanup via background task
- Respects TTL for queued offline messages

How Implemented:

- Background cleanup runs every 5 minutes
- Deletes messages where expires_at < current_time
- Also cleans up expired sessions and challenges

Limitations:

- Best-effort: cleanup may be delayed
- Cannot prevent client-side screenshots
- Relies on server behaving correctly

Code Location:

- Server: `server/main.py` - cleanup background task
- Database: `server/db.py` - `cleanup_expired_messages()`
- Policy: `server/main.py` - startup event triggers cleanup

A.5 Friends / Contacts

(R13) Friend Request Workflow

Implementation Summary:

- Users send friend requests by username
- Requests must be accepted/declined (not instant adding)
- Mutual consent required for contact relationship

How Implemented:

- `POST /friend-requests` to send requests

- `GET /friend-requests` to view pending
- `POST /friend-requests/{id}/respond` to accept/decline

Related Commands:

```
# Send friend request
im> send-request bob

# View pending requests
im> pending

# Accept request (ID 1)
im> respond 1 accept
```

Code Location:

- Server: `server/main.py` - friend request endpoints
- Database: `server/db.py` - contacts and friend_requests tables
- Client: `client/cli.py` - friend request commands

(R14) Request Lifecycle

Implementation Summary:

- Senders can cancel pending requests
- Receivers can accept or decline
- Both parties can view pending status

How Implemented:

- Request states: pending, accepted, declined, cancelled
- `DELETE /friend-requests/{id}` for cancellation
- Status updates reflected in real-time

Related Commands:

```
# Cancel sent request
im> cancel-request 1

# Decline received request
im> respond 1 decline

# View all pending requests
im> pending
```

Code Location:

- Server: `server/main.py` - request lifecycle management
- Database: `server/db.py` - friend_requests table status tracking
- Client: `client/cli.py` - request management commands

(R15) Blocking / Removing

Implementation Summary:

- Users can block other users
- Blocked users' messages and requests are ignored
- Can remove friends (deletes contact relationship)

How Implemented:

- Block list stored in database
- Server filters messages from blocked users
- Remove friend deletes contact relationship

Related Commands:

```
# Block a user
im> block bob

# View blocked users
im> blocked

# Unblock a user
im> unblock bob

# Remove friend
im> remove-friend bob
```

Code Location:

- Server: `server/main.py` - block/unblock endpoints
- Database: `server/db.py` - blocked_users table
- Client: `client/cli.py` - blocking commands

(R16) Default Anti-spam Control

Implementation Summary:

- Non-friends cannot send chat messages by default
- Only friend requests allowed from non-contacts
- Rate limiting prevents spam

Implementation Justification:

- Prevents unsolicited messages
- Reduces spam and abuse potential
- Encourages proper friend request workflow

How Implemented:

- Server checks contact relationship before accepting messages
- `POST /messages/send` validates sender-receiver are contacts
- Rate limiting on message endpoints

Code Location:

- Server: `server/main.py` - message sending validation
- Policy: `server/main.py` - contact relationship checks
- Security: `server/rate_limiting.py` - anti-spam measures

A.6 Message Delivery Status

(R17) Minimum Delivery States

Implementation Summary:

- **Sent**: Message successfully submitted to server
- **Delivered**: Message forwarded to recipient's active connection

How Implemented:

- Server returns "sent" status after storing message
- WebSocket push marks message as "delivered"
- Status stored in database messages table

Related Commands:

```
# Status is automatic, no direct command
# View message status in conversation
im> open 1
```

Code Location:

- Server: `server/main.py` - message status tracking
- Database: `server/db.py` - messages.status field
- WebSocket: `server/ws_manager.py` - delivery notifications

(R18) Define "Delivered" Semantics

Implementation Summary:

- **Option A (Implemented)**: Delivered = server placed ciphertext in recipient's queue or forwarded to active connection
- Simple, efficient, and sufficient for HbC model

Implementation Details:

- Server marks message as "delivered" after:
  - Storing in database (for offline users)
  - Sending via WebSocket (for online users)
- No client acknowledgment required

Why Option A:

- Simpler implementation
- Sufficient for HbC server model
- Reduces metadata leakage (no ack messages)
- Better performance (fewer round trips)

Code Location:

- Server: `server/main.py` - delivery status logic
- WebSocket: `server/ws_manager.py` - online delivery
- Database: `server/db.py` - status updates

(R19) Metadata Disclosure Statement

Implementation Summary:

- Server learns: message timing, delivery status, online/offline status
- Metadata: sender/receiver usernames, timestamps, message size
- Limited exposure due to E2EE of content

Server Learns:

- **From "Delivered"**: When recipient is online, approximate online times
- **From Message Storage**: Communication patterns, frequency, timing
- **From Status Updates**: When users are active vs offline

Mitigation:

- Content remains encrypted (E2EE)
- No message content exposed
- Metadata leakage is unavoidable under HbC model

Code Location:

- Analysis: `report_cn.md` and `report_en.md` - metadata analysis sections
- Server: `server/main.py` - all endpoints that log/track metadata

A.7 Offline Messaging (Ciphertext Store-and-Forward)

(R20) Offline Ciphertext Queue

Implementation Summary:

- Server queues messages for offline recipients
- Messages stored as encrypted ciphertext
- Delivered when recipient comes online

How Implemented:

- Messages stored in database regardless of online status
- WebSocket connection established on client login
- Server pushes queued messages via WebSocket

Related Commands:

```
# Automatic behavior
# Messages queued when recipient offline
# Delivered on next login
im> login bob StrongPass123  # Triggers delivery of queued messages
```

Code Location:

- Server: `server/main.py` - message storage and WebSocket delivery
- Database: `server/db.py` - messages table with offline queue flag
- WebSocket: `server/ws_manager.py` - queued message delivery

(R21) Retention and Cleanup

Implementation Summary:

- Messages retained until delivery or TTL expiration
- Background cleanup removes expired messages
- TTL respected for queued ciphertext

Retention Policy:

- **Normal Messages**: Retained until delivered
- **TTL Messages**: Deleted after expires_at time
- **Cleanup Frequency**: Every 5 minutes
- **Max Age**: TTL-based, no permanent storage

How Implemented:

- `cleanup_expired_messages()` runs periodically
- Deletes messages where expires_at < now()
- Also cleans up orphaned offline messages

Code Location:

- Server: `server/main.py` - cleanup background task
- Database: `server/db.py` - cleanup functions
- Policy: `server/main.py` - startup and periodic cleanup

(R22) Duplicate/Replay Robustness

Implementation Summary:

- Clients handle duplicate messages gracefully
- Replay protection prevents accepting old ciphertext as new
- Robust handling of network retries and reconnections

How Implemented:

- Replay cache prevents duplicate processing
- Duplicate delivery detection for WebSocket reconnections
- Message IDs prevent confusion

Robustness Features:

- Idempotent message processing
- Duplicate detection and ignore
- Replay token validation

Code Location:

- Client: `client/e2ee.py` - replay protection
- Tests: `tests/test_replay_protection.py` - comprehensive tests
- WebSocket: `client/ws_client.py` - duplicate handling

A.8 Conversation List & Unread Counters

(R23) Conversation List

Implementation Summary:

- Shows list of conversations with contacts
- Ordered by most recent activity
- Includes last message time and preview

How Implemented:

- Database query for conversations with message activity
- Sort by last message timestamp (descending)
- Display contact name and last message preview

Related Commands:

```
# Show conversation list
im> conversations

# Shows: ID, Contact, Last Message, Time, Unread Count
```

Code Location:

- Server: `server/main.py` - `GET /conversations` endpoint
- Database: `server/db.py` - conversation query functions
- Client: `client/cli.py` - `conversations()` command

(R24) Unread Counters

Implementation Summary:

- Maintains unread count per conversation
- Updates when messages are marked as read
- Displayed in conversation list

How Implemented:

- Database tracks is_read flag per message
- Unread count = COUNT(*) WHERE is_read = 0
- Mark as read updates database flags

Related Commands:

```
# Mark conversation as read
im> mark-read 1

# View unread counts in conversation list
im> conversations
```

Code Location:

- Server: `server/main.py` - `POST /messages/{id}/ack` endpoint
- Database: `server/db.py` - message read status tracking
- Client: `client/cli.py` - unread count calculation

(R25) Paging / Incremental Loading

Implementation Summary:

- Basic pagination for message history
- Loads messages in batches to avoid memory issues
- Supports loading older messages on demand

How Implemented:

- `GET /messages/{conversation_id}?limit=N&offset=M`
- Default loads 20 most recent messages
- Can specify limit and offset for pagination

Related Commands:

```
# Open conversation with pagination
im> open 1          # Loads 20 most recent
im> open 1 50       # Loads 50 messages
im> open 1 20 40    # Loads 20 messages starting from offset 40
```

Code Location:

- Server: `server/main.py` - `GET /messages/{conversation_id}` endpoint
- Database: `server/db.py` - paginated message queries
- Client: `client/cli.py` - pagination support in `open_conversation()`

Summary

Total Requirements: 25
Implemented: 25
Partially Implemented: 0
Not Implemented: 0

All requirements (R1-R25) have been fully implemented in the Obsidian instant messaging system. The implementation provides a complete end-to-end encrypted messaging platform with modern security features, user management, and robust message delivery mechanisms.

Key Security Achievements:

- End-to-end encryption using X25519 + HKDF-SHA256 + AES-GCM
- Server cannot access message content
- Replay attack protection

- Man-in-the-middle attack prevention
- Secure user authentication with 2FA
- Privacy-preserving offline messaging

Implementation Quality:

- Comprehensive test coverage for security features
- Clean separation of concerns between client and server
- Well-documented protocol and security analysis
- User-friendly CLI interface with all required features

---

8. Conclusion

8.1 Project Achievement Summary

This project successfully implemented an end-to-end encrypted instant messaging system named **Obsidian**, achieving all preset goals:

Technical Achievements:

- **Complete End-to-End Encryption Implementation**: Modern cryptographic solution based on X25519 + HKDF-SHA256 + AES-GCM
- **Server Cannot Access Plaintext**: Client-side encryption ensures message content is completely confidential to server
- **Secure Identity Authentication**: Password + TOTP dual authentication mechanism
- **Real-time Communication Capability**: WebSocket-supported real-time message push
- **Complete Social Features**: Friend management, contact system, blacklist functionality

Security Achievements:

- **Replay Attack Prevention**: Replay token mechanism effectively protects against replay attacks
- **Man-in-the-Middle Attack Prevention**: TOFU trust model detects key tampering
- **Transport Security**: TLS encryption protects network transmission
- **Integrity Protection**: AES-GCM authenticated encryption prevents message tampering

Academic Value:

- Provides complete end-to-end encryption system implementation reference
- Validates practical application of modern cryptographic protocols
- Provides practical cases for network security education
- Detailed security analysis and testing methods

8.2 Technical Contributions

Innovation Points:

1. **Academic Transparent Implementation**: Complete open-source end-to-end encryption system, suitable for academic research and teaching
2. **Simplified but Not Simple**: Simplifies design while ensuring security,便于 understanding and learning
3. **Complete Testing Validation**: Provides detailed security test cases and verification methods
4. **Threat Model Analysis**: Systematic security threat analysis and protection design

Practical Value:

1. **Deployable Prototype**: Complete system that can be directly deployed and run
2. **Educational Tool**: Suitable for practical teaching of network security courses
3. **Research Foundation**: Provides basic platform for further research
4. **Standard Compliance**: Follows RFC standards and best practices

8.3 Project Significance

The successful implementation of this project demonstrates:

1. **Technical Feasibility**: Complete feasibility of end-to-end encryption technology in academic projects
2. **Security Assurance**: Security goal of servers not being able to obtain plaintext can be achieved through correct design
3. **Educational Value**: Complete implementation provides valuable practical resources for network security education

The **Obsidian** project is not just a technical implementation, but also a practical exploration of modern digital privacy protection, demonstrating how to achieve strong security protection while ensuring usability.